Tools, Not Prompts: Revisiting LLM-Based Legacy PHP Modernization with an Agent Harness

Ianiv Schweber - Just a random guy on the Internet

November 2, 2026

Abstract

A recent study evaluated five large language models (LLMs) on modernizing 100 WordPress 4.0 PHP files to PHP 8.3. Given one prompt per file and no tools, tests, or runtime feedback, every model ended with fewer behaviorally compatible files than leaving the code unchanged (83 of 100). The study scopes this finding to the first answer of a single-file workflow, and it anticipates both newer models and execution feedback. Its headline is nonetheless easy to repeat as evidence that LLMs cannot produce working migrations. We test that broader reading, not the authors' scoped claim, with the study's own released harness (630 hidden PHPUnit checks and Rector's PHP-version rules) and Claude Opus 5.5. Under the study's single-shot protocol and limits, Opus 5.5 produces 93 compatible files and breaks one originally compatible file. As a coding agent with PHP 8.3, the file's checks, a linter, and Rector, scored on a pristine copy of the harness, it produces 99 compatible files with zero regressions. Both conditions exceed the unchanged code significantly (exact McNemar, Holm-adjusted $p \leq 0.025$); the difference between them is not significant on a single run. The agent condition implements the staged, execution-feedback workflow that the original study recommends, and it has access to the very checks that score it, so it measures what a developer with a test suite can achieve. The one file the agent does not pass is blocked by a defect in that file's test environment. We validate our harness by reproducing the published baseline, the Sonnet 4 and Rector results, and, with a reconstructed rule mapping, the study's necessity breakdown and sensitivity analyses.

1 Introduction

Much of the web still runs PHP that was written for PHP 5. PHP 7 and 8 removed or redefined many of the constructs that code depends on: `each()`, `create_function()`, curly-brace string offsets, PHP 4-style constructors, and lenient handling of `null` and non-countable values. Moving such code to a supported runtime is costly, repetitive, and easy to get subtly wrong, which makes it a natural target for automation.

The study we revisit [1] asks whether LLMs can do this work safely. It samples 100 files from WordPress 4.0 and asks five models to migrate each file to PHP 8.3 in a single prompt. It then scores the outputs with 630 hidden PHPUnit checks and with Rector's PHP-version rules. Its headline finding is negative. Under PHP 8.3, 83 of the 100 unmodified files are behaviorally compatible, while the best model reaches 59. The rule-based tool Rector reaches 82.

That finding is accurate for the condition the study measures, and the study is careful to say so. It evaluates "the first answer, not an agent's result after iterating on errors" (§I). It notes that "newer models may avoid specific errors, but the evaluation design applies unchanged" (§V). And it recommends treating the first answer as a draft: a staged workflow of lint, load with dependencies, then check behavior, after which "execution feedback could then drive repair" (§V). A headline such

as "every evaluated LLM left fewer working files than it started with" is nonetheless easy to repeat without these qualifications, as evidence that LLMs cannot migrate legacy code. Within days of posting, one article headlined the study "AI Legacy Code Modernization Breaks More Working Code Than It Fixes" [2], although its body does note that each model got "one file, one prompt and no chance to run its own output." This paper tests that popular reading, not the authors' claim.

The popular reading depends on the two things the study deliberately sets aside. The first is model generation. The models evaluated in [1] were released between December 2024 (Llama 3.3 70B [3]) and September 2025 (GPT-5 Codex [4]). Claude Sonnet 4 dates from May 2025 [5], and both Gemini 2.5 models became generally available in June 2025 [6]. The study's tool documentation was accessed in August 2026 and the paper was posted at the end of October 2026, so the models were roughly one to two years old when it appeared. The second is execution feedback. In the study, a model sees one file and nothing else, and it never runs, lints, or tests its output, whereas coding agents iterate against an interpreter and a test suite. We therefore ask how much of the gap a newer model closes under the identical protocol, and what the study's own recommended workflow achieves.

We answer them with the study's own artifacts. We pose three research questions:

- **RQ1 (Reproduction).** Can the released harness be run independently and reproduce the published baseline and model results?
- **RQ2 (Model).** Under the identical single-shot protocol, how does a current frontier model (Claude Opus 5.5) compare with the models in the original study and with the unchanged code?
- **RQ3 (Harness).** When the same model runs as a coding agent with an interpreter, the file's checks, a linter, and Rector (the staged workflow recommended in [1]), how close does it come to a complete and behavior-preserving migration?

Our contributions are:

1. An independent reproduction of the study's evaluation on macOS with PHP 8.3 in a Linux VM. It matches the no-change baseline check for check and the published Sonnet 4 and Rector results exactly. With a reconstructed rule-to-necessity mapping it also reproduces the study's resolution-by-necessity table and its sensitivity analyses.
2. A single-shot replication with Opus 5.5 that keeps the study's prompt, extractor, and limits. Opus 5.5 alone exceeds the no-change baseline (93 vs. 83 compatible files).
3. An agent harness, scored on a pristine copy of the harness, that reaches 99 of 100 compatible files with zero regressions. We trace the remaining file to a defect in its test environment.
4. A released replication package with all outputs, scores, each agent session's final report and run statistics, the re-analysis, and a contamination check against later WordPress releases.

2 Background and Related Work

Rule-based migration. Rector [7] rewrites PHP abstract syntax trees using rule sets keyed to language versions, such as `UP_TO_PHP_83`. Its rules are local and mechanical, so they rarely change behavior that already works. For the same reason, they cannot repair failures that depend on semantics: a `count()` on `null`, which now throws, or a comparison whose result changed in PHP 8. The original study uses Rector both as a baseline and as the source of its legacy-pattern metric.

Evaluating behavior, not text. TransCoder [8], an unsupervised neural translator between C++, Java, and Python, introduced *computational accuracy*: a translation counts as correct only if it produces the reference outputs on unit tests. Its motivation is the gap that the hidden checks of [1] also address. Only 3.1% of TransCoder's C++-to-Java translations matched the reference text, yet 60.9% passed the tests. TransCoder also used the tests to choose among beam-search candidates, which raised computational accuracy by up to 33.7 points (Java to Python). Choosing by execution is the simplest form of the feedback our agent condition provides.

LLMs silently change behavior when translating. Pan et al. [9] found that 2.1% to 47.3% of LLM code translations were correct, depending on the model. Most failed translations (77.8%) did not compile, but 13.4% of them compiled and ran yet failed tests. That silent change of behavior is the failure mode the hidden checks of [1] detect. Re-prompting with the error output, plus a failing input and expected output for test failures, raised GPT-4's success rate by 12.33% after one round and the four repaired models' by 5.5% on average, although some repairs turned one kind of bug into another. Our preservation metric tracks this risk: repair in a loop must not cost regressions.

Execution feedback, not iteration, drives the gain. Self-debugging [10] lets a model revise its code from feedback over up to 10 turns. On C++-to-Python translation it raised Codex from 80.4% to 92.5% when the feedback included unit-test results and the model's own explanation of its code, but only to 83.9% when the model could explain or trace its code without executing the tests. This split between generation and execution feedback mirrors our RQ2 and RQ3 at function scale.

Agents on repository-scale tasks. SWE-bench [11] judges a patch by tests that must go from failing to passing and tests that must keep passing. These are the same repair and preservation criteria that [1] uses. Its baselines generate a patch in one pass from retrieved files, and the best resolved 1.96% of issues. With the same model (GPT-4 Turbo), SWE-agent [12] raised the resolve rate on SWE-bench Lite from 2.67% for that single-pass baseline to 18.0%, by letting the model navigate the repository, edit files, and run code. Its ablations credit a linter that rejects syntactically invalid edits with 3.0 of those points. Our agent's `tools/lint` serves the same role.

Evaluation condition as a variable. Our study treats the presence of execution feedback as an experimental factor, separate from the model. To isolate the model's contribution we hold the prompt, file, and scoring fixed (RQ2). We then add the tool loop with the model held fixed (RQ3).

3 The Original Study

Dataset. The study [1] draws on WordPress 4.0's 482 PHP files. It keeps the 397 that have Rector findings and selects 100 by a deterministic, stratified procedure that covers all rule types. By size, 31 files have at most 200 lines, 31 have 201–500, 26 have 501–1,000, and 12 exceed 1,000 lines. Together they total 54,321 lines.

Hidden checks. The study defines 630 PHPUnit checks: 109 reachability checks, 441 output or state checks, and 80 semantic-fingerprint checks. Each test file targets one subject. It runs the subject in a sandbox built from the WordPress 4.0 tree plus per-file stubs, under PHP 8.3 with `E_ALL`, and it counts deprecation notices as failures. Unmodified, the files pass 531 checks. The harness report in the study's replication package [13] attributes the 99 failures to removed

or changed PHP features: 59 parse errors, 21 output mismatches, 8 undefined symbols, 6 type or signature errors, and 5 fatal runtime errors.

Metrics. For a migrated variant, each check falls into one of four transitions relative to the original: preserved pass, regression, repair, or unrepaired failure. The study reports:

- **Compatible files:** files whose checks all pass.
- **Preservation:** preserved passes ÷ 531, the original passes that still pass.
- **Repair:** repairs ÷ 99, the original failures that now pass.
- **Legacy-pattern resolution:** Rector `Php*` findings in the original that no longer fire after migration. A finding is a (file, rule type) pair; there are 511 across 44 rule types. Rector runs on each file in isolation, and unparseable output resolves nothing.

Necessity. The study groups Rector's findings by what happens if the construct is left unchanged under PHP 8.3. *Required* constructs fail or change behavior (for example, curly-brace string offsets or PHP 4-style constructors), *deprecated* ones run but raise notices, and *optional* ones remain valid. Only 37 of the 511 findings are required, against 85 deprecated and 389 optional, and optional changes make up 82–87% of what each model resolves. The study's central observation is that most broken files needed no required change: the LLMs break 36–60% of the compatible files that contain no required finding. In its words, "resolution measures what changed, not whether it should have."

Condition. Each model receives a fixed prompt that asks a senior PHP developer to migrate the file to PHP 8.3 while preserving functional equivalence, followed by the full file. Temperature is 0.2, except for GPT-5 Codex, whose API does not expose it; reasoning and thinking settings stay at provider defaults. Requests allow up to 64,000 output tokens and 600 seconds. The study keeps the first complete response, regenerating incomplete ones, and extracts code between `// MIGRATION_START` and `// MIGRATION_END` markers. The model sees no other files, tests, or errors.

Statistics and sensitivity. File-level compatibility is compared with two-sided exact McNemar tests under Holm–Bonferroni correction, and effect sizes are paired differences in compatible files with 95% percentile bootstrap intervals (10,000 file resamples). Two sensitivity analyses recount compatible files without the 80 fingerprint checks and with a lint fallback, which substitutes the original for any output that `php -l` rejects.

Reported results. Every LLM falls below the unchanged code on compatible files: Sonnet 4 reaches 59, GPT-5 Codex 58, Gemini 2.5 Pro 55, Gemini 2.5 Flash 50, and Llama 3.3 70B 33, against Rector's 82. With the lint fallback the best LLM, GPT-5 Codex, reaches 69 (Sonnet 4: 67), still below the unchanged code. The study does not score Rector's own resolution, which would be circular. The full figures appear alongside ours in Table 2.

4 Methodology

4.1 Environment and harness reproduction

We download the study's replication package [13] and run it unchanged except for the runner. The package's PowerShell runner sets three environment variables and calls PHPUnit, so we

port it to a 10-line bash script. PHP runs in a Linux VM under Apple's `container` (v1.6.1, Virtualization.framework) from the official `php:8.3-cli` image, which provides PHP 8.3.36. It includes the bcmath, curl, dom, mbstring, pdo, simplexml, tokenizer, xml, and zip extensions, with `error_reporting=E_ALL`. Tools come from the package's lock files: PHPUnit 9.6.34 and Rector 2.1.0.

We validate the reproduction three ways before running any model (RQ1):

1. Run the unchanged originals and diff the per-check testdox output against the package's `OriginalTestResults.txt`.
2. Re-score the package's archived Sonnet 4 outputs end to end and compare with the published summary.
3. Run Rector on every original and count `Php*` findings against the package's metadata.

4.2 Single-shot condition (RQ2)

We reuse the study's prompt template verbatim, insert the full file, and extract code with a line-for-line port of its extractor. Like the original, we read files as UTF-8 with undecodable bytes ignored. Calls go to Claude Opus 5.5 through the Claude Code CLI in print mode, with every tool disabled, a one-line system prompt, no user or project settings, and a 64,000-token output cap. As in [1], we keep the first complete response and regenerate incomplete ones, up to three attempts.

Two details differ from a raw API call. The CLI does not expose temperature, so sampling uses the default, as the original study did for GPT-5 Codex, whose API does not expose it either. Opus 5.5 also uses adaptive extended thinking by default, which we leave on: on file 004 it spent 3,786 thinking tokens.

The CLI's output cap applies per assistant turn, and the CLI continues a response that reaches it. One response did so. For file 014 (`module.tag.id3v2.php`, 134 KB, the largest subject), the model used 84,406 output tokens (18,611 of them thinking) over two turns and 619 s. That exceeds the study's limits of 64,000 tokens and 600 seconds. Opus 5.5's migrated code alone is about 65,800 tokens, so its response could not fit within those limits, and under the original protocol this file would have had no complete Opus response (Sonnet 4's shorter output for this file did fit, and passes all six checks). We therefore count file 014 as failed in our primary, *strict* single-shot result (93 compatible files), and we report the response as delivered (94) alongside it. No other response exceeded 62,554 output tokens or 417 s, or needed a second turn.

4.3 Agent condition (RQ3)

Each file gets its own workspace. It is a copy-on-write clone of the harness with every archived model output, result file, and report removed, so the agent cannot see how other models did. The subject directory starts as a copy of the originals. The agent may edit exactly one file, the migrated copy of its subject.

Five tools run inside the PHP 8.3 VM:

- `tools/test` runs the file's checks against the migrated copy.
- `tools/test-original` runs the same checks against the original.
- `tools/lint` runs `php -l`.
- `tools/rector` lists the remaining `Php*` findings, with the diff Rector would apply.

- `tools/php` runs the PHP CLI for scratch scripts.

The agent can read the checks, the stubs, and the full WordPress 4.0 tree. Its instructions restate the study's goal of migrating to PHP 8.3 while preserving observable behavior. They add a definition of done: every check passes, no Rector finding remains unless applying it would change behavior or call compatibility (with a reason for each one kept), and lint is clean. They forbid edits outside the target, special-casing test inputs, detecting the test environment, and deleting uncovered functionality.

The agent is Claude Code (v2.1.316) in headless mode with model `claude-opus-5-5` and default effort. It runs in `dontAsk` permission mode, with an allowlist of file tools, the five tools, and read-only shell commands. Web access, `curl`, and `wget` are denied, and user settings, hooks, and MCP servers are not loaded. Each file gets one session with no turn limit, and six sessions run in parallel.

The task prompt names the target file, its original, and its test file. It was also meant to give the file's path in the WordPress tree, but a lookup bug left that path blank in 85 of the 100 prompts ("It is the WordPress file `"`."). The lookup matched `config/file-map.php` keys exactly, and from file 013 on most keys carry a `.php` suffix that our subject IDs do not. The 15 prompts with a path are exactly the 15 files whose key has no suffix (001–012, 014, 016, and 030). The omission removes information rather than adding any, and every agent still had the target, its tests, and the full tree.

4.4 Run history

We report every launch, including those that produced no model output.

- **Agent, file 004.** Run first, alone, as a pilot. Its first launch hung while cloning the workspace, because of 21 GB of leftover test sandboxes, and was killed before the agent started. The second launch is the shipped session. The task prompt did not change afterwards.

- **Agent, batch.** The other 99 files ran in one logged batch. Files 003 and 008 failed before their agents started, because the path lookup also matched example comments in `file-map.php` and broke prompt construction. We anchored the lookup and ran those two once more. No file was run more than once with a model session, and no completed session was discarded.

- **Single-shot.** A pilot call on file 004 was kept as that file's response. The batch started with 4 parallel calls and was restarted with 8, keeping every completed response. File 062's worker crashed on an encoding error before making any call, and we fixed it by reading files exactly as the original does (UTF-8, undecodable bytes ignored). File 014's first two responses were discarded because our extractor read only the last turn of each multi-turn response, which we then fixed. All three 014 responses spanned two turns and ran for about ten minutes, beyond the study's limits, which is why the strict result counts the file as failed.

4.5 Scoring

Both conditions are scored by the same pipeline. It builds a fresh copy of the released harness, copies in only each run's migrated target files, and runs all checks. It then runs Rector on each migrated file in its own directory, matching the study's single-file isolation. Because tests, stubs, and configuration come from the pristine copy, an agent's edits outside its target cannot affect its score. After each session we diffed the agent's tests, configuration, and other subject files against the pristine harness. After the runs we extended that check to every file in each retained workspace (the WordPress tree, `vendor/`, `phpunit.xml`, Composer files, tests, configuration, originals, and

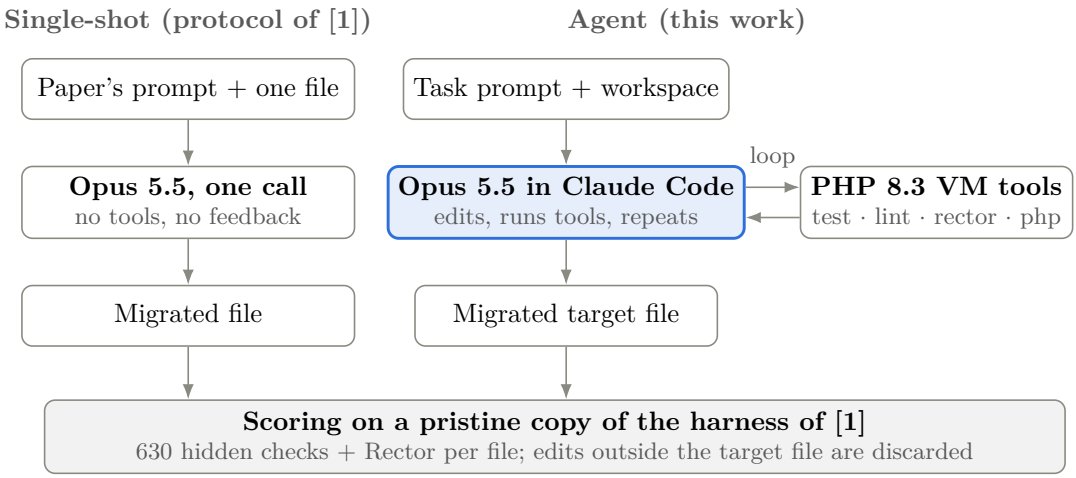


Figure 1: The two experimental conditions and the shared scoring pipeline. Both receive the same file and are scored by the same pipeline; only the agent can run code against the checks before submitting.

the other subjects), excluding only the target, `scratch/`, `tools/`, and `build/`. We count the 630 checks of the study, which, like the study, excludes the package's setup smoke test (Section 5.1).

5 Results

5.1 RQ1: The harness reproduces the published results

The reproduction matches the study wherever both can be compared (Table 1).

The package contains 631 checks, and both its results file and our run pass 532 of them. The extra check is a setup smoke test that the study rightly excludes, giving its 630 and 531. The package's archived junit files for Sonnet 4 and Rector are stale (Sonnet 4's records 467 passed checks, Rector's 32 compatible files), so the re-analysis uses our re-scores of their outputs, which reproduce the published rows exactly. We also re-scored the other four models' archived outputs: they reproduce the published compatible-file counts, with check totals within three of the published ones (Flash 394, Pro 448, Codex 470, Llama 337), plausibly from the PHP patch-level difference. For those four the re-analysis uses the package's junit files, which match the published totals. Rector's count depends on context. Run over the whole directory, Rector reports 61 extra `RemoveExtraParametersRector` and 7 extra `AddOverrideAttributeToOverriddenMethodsRector` findings, because it can resolve signatures defined in other files. It also misses 5 that it finds in isolation. Only per-file isolation reproduces 511, so we use it throughout.

5.2 Main results

Figure 2 recreates the file-level view of [1] and adds our two conditions. Every model in [1] breaks between 32 and 51 of the 83 working files. Opus 5.5 breaks one file single-shot and none as an agent, and the agent fixes 16 of the 17 originally incompatible files. The seventeenth is file 009 (Section 5.6).

Table 1: Reproduction checks against the released package.

Check	Published	Reproduced
Originals: per-check testdox outcome	`OriginalTestResults.txt`	identical, 631 of 631 lines
Originals: checks passed	531 of 630 (paper text)	532 of 631 (package); 531 of 630 without the smoke test
Originals: compatible files	83	83
Sonnet 4 archived outputs: per-check testdox	package `testdox.txt`	identical
Sonnet 4: compatible / preservation / repair	59 / 76.3% / 68.7%	59 / 76.3% / 68.7%
Sonnet 4: Rector findings resolved	272	272
Rector outputs: compatible / checks / regressions / repairs	82 / 581 / 12 / 62	82 / 581 / 12 / 62
Rector `Php*` findings on originals	511	511 (per-file isolation); 574 (whole directory)
Findings required / deprecated / optional	37 / 85 / 389	37 / 85 / 389 (reconstructed mapping)
Resolution by necessity, five LLMs (Table IV of [1])	15 rates, 5 introduced counts	all match
Required-file split: broken / fixed files	LLMs 5–7 / 6–9 (Llama 1); Rector 1 / 6	LLMs 5–7 / 6–9 (Llama 1); Rector 1 / 6
No-fingerprint and lint-fallback compatible files	Table III	all match
Re-score of all five LLMs' archived outputs: compatible files	59 / 50 / 55 / 58 / 33	all match; check totals within 3

5.3 RQ2: The model alone exceeds the unchanged code

With the study's prompt, limits, and no feedback, Opus 5.5 produces 93 compatible files (94 as delivered). That is 10 more than the unchanged code, 11 more than Rector, and 34 more than the best model in [1]; it is also 24 more than the best lint-fallback result of that study (GPT-5 Codex, 69). It breaks one check across 531 original passes, a semantic-fingerprint check for nav-menu sorting in file 054, and repairs 81 of the 99 failing checks (87 as delivered). Of the 99 responses within the study's limits, all were complete on the first attempt.

The strict single-shot result fails checks in 7 files. Apart from 009 (Section 5.6) and the over-limit 014, the failures are narrow: 3 of 7 checks for `class-wp.php` request parsing (007), one AC3 lookup table (016), POP3's empty-mailbox marker (021), JSON encoding of primitives (030), and the nav-menu regression (054). On legacy patterns, single-shot Opus 5.5 resolves 66.3% of findings (67.5% as delivered) and introduces 26. That is more than Sonnet 4 and GPT-5 Codex and less than Gemini 2.5 Pro (72.8%). Only 21 files leave both all checks passing and no Rector findings.

5.4 RQ3: With tools and the checks, the migration is behavior-preserving

As an agent, Opus 5.5 breaks none of the 531 passing checks and repairs 93 of 99 failing ones. All 6 unrepaired checks belong to file 009. It passes every file the single-shot run left failing except 009 and raises resolution from 66.3% to 92.4%, introducing 4 new findings. Altogether 68 files finish with all checks passing and no remaining Rector findings.

The agent's instructions allow a finding to stay when applying it would change behavior or call compatibility. Of the 511 original findings, 39 remain, and the agent introduced 4 new ones. The

Table 2: Results on the 100 files, 630 checks, and 511 Rector findings. Rows above the rule are from [1] (Tables II and IV). Following that study, Rector's own resolution is not scored, because it would be circular. Single-shot (strict) counts file 014 as failed because its response exceeded the study's output limits (Section 4.4); "as delivered" scores that response.

Approach	Compatible files	Checks passed	Regressions	Repairs	Rector findings resolved	Rector findings introduced
Unchanged originals	83	531	–	–	0 (0%)	–
Llama 3.3 70B	33	337	215	21	156 (30.5%)	18
Gemini 2.5 Flash	50	393	198	60	273 (53.4%)	17
Gemini 2.5 Pro	55	446	147	62	372 (72.8%)	21
GPT-5 Codex	58	467	122	58	297 (58.1%)	20
Claude Sonnet 4	59	473	126	68	272 (53.2%)	17
Rector 2.1.0	82	581	12	62	–	–
Opus 5.5, single-shot (strict)	**93**	**611**	**1**	**81**	**339 (66.3%)**	26
as delivered	94	617	1	87	345 (67.5%)	27
Opus 5.5, agent	**99**	**624**	**0**	**93**	**472 (92.4%)**	4

resulting 43 findings fall under 10 rules, most of them optional rewrites (Figure 3). The agent's definition of done asked for no remaining Rector findings unless keeping one was justified. That instruction targets the aggregate resolution metric that [1] argues is misleading, so we read the agent's resolution together with the necessity breakdown in Section 5.5 rather than on its own.

In 30 of the 32 files that keep findings, the agent's final report names every kept rule and gives a reason. The reports for 010 and 022 name some but not all. The reasons fall into three groups:

- **Optional modernization.** Constructor promotion, `switch` to `match`, and first-class callables are valid PHP 8.3 without them and raise no notice. Agents declined them where they would reorder properties visible to `get_object_vars()`, or where `match`'s strict comparison would differ from `switch`'s loose one.
- **Compatibility shims.** Seven `Php4ConstructorRector` findings remain where the agent added `__construct` but kept the old same-name constructor as a forwarding wrapper. WordPress subclasses such as `wp_xmlrpc_server` still call `$this->IXR_Server(...)`.
- **Defensive casts.** The agents declined `NullToStrictStringFuncCallArgRector` casts on request superglobals whose values are always strings at runtime.

In all 100 retained workspaces, no file outside the target differs from the pristine harness, including the WordPress tree, `vendor/`, and `phpunit.xml` (`runs/opus55/tamper_full.txt`). All 100 sessions hit at least one permission denial, 123 in total. All but one were compound shell commands (chained with `&&` or `;`, piped, or using heredocs or redirects), which the per-command allowlist does not match; 52 of them were in-place `perl` edits. Despite the denials, every session ended with a migrated file and a final report.

Cost and effort. The agent run cost $39.66 in total: a median of $0.30, 14 turns, and 73 s per file, with at most $1.56, 59 turns, and 359 s. The single-shot run cost $26.92, a median of $0.19 per file. After the single pilot session on file 004, the batch of the other 99 sessions took about 30 minutes of wall-clock time at six sessions in parallel.

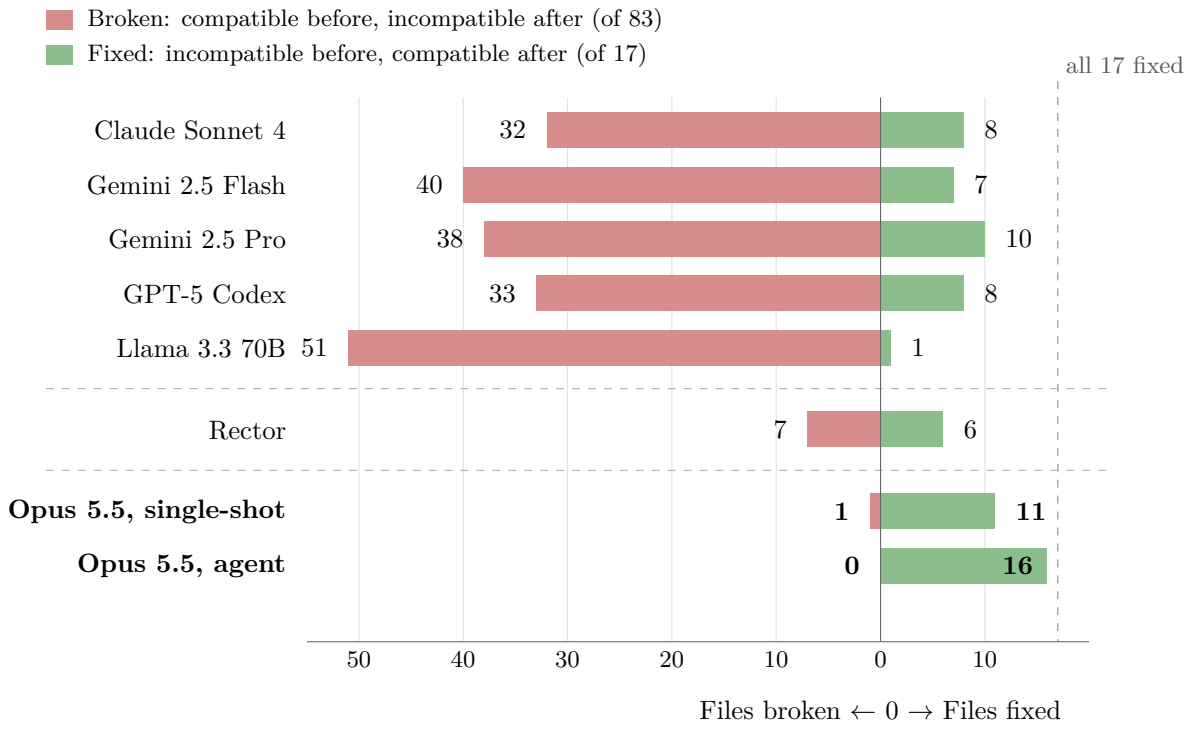


Figure 2: Files broken and fixed per approach. The top five rows are the models of [1], then Rector, then our two conditions (single-shot strict; as delivered it fixes 12). Rows from [1] are recomputed from the package's `per_file.csv` and match the paper's figure; Opus rows come from `runs/*/score/per_file.csv`.

5.5 Necessity, sensitivity, and significance

The original package does not include the rule-to-necessity mapping that [1] uses (its source notes the mapping as incomplete), so we reconstructed one from the study's definitions. Thirteen rules are *required*, among them curly-brace offsets, PHP 4 constructors, `ereg`, `create_function`, `each`, case-insensitive constants, and integer needles. Three are *deprecated*: null-to-string arguments, `utf8_encode`/`decode`, and `final private`. The rest are optional. The mapping reproduces the published group sizes, all 15 of the study's per-model resolution rates, and its five introduced counts (Table 1); it is in `runs/reanalysis/necessity_mapping.csv`. Table 3 applies it, together with the study's sensitivity analyses and tests, to both Opus conditions.

The original's central finding does not recur. The study's models break 26–44 of the 73 compatible files that need no required change. Single-shot Opus 5.5 breaks one of them (054), and the agent breaks none. Neither Opus condition breaks a compatible file that contains a required finding. Optional changes still dominate what Opus resolves (82.3% of resolved findings single-shot, 77.5% for the agent), but unlike the earlier models' optional changes, they rarely cost behavior.

Required findings. On required findings the agent resolves 75.7%, slightly below the 78.4% of Sonnet 4 and both Gemini models, although every file in which one still fires passes its checks. Of the nine that still fire, five are PHP 4 constructors where the agent added `__construct` and kept the old name as a forwarding wrapper, so the rule still matches the wrapper. The other four the agent's reports describe as false positives or as fixed at the failing site. In 002, applying `SensitiveConstantNameRector` would rename constants that the file defines with mixed case. In

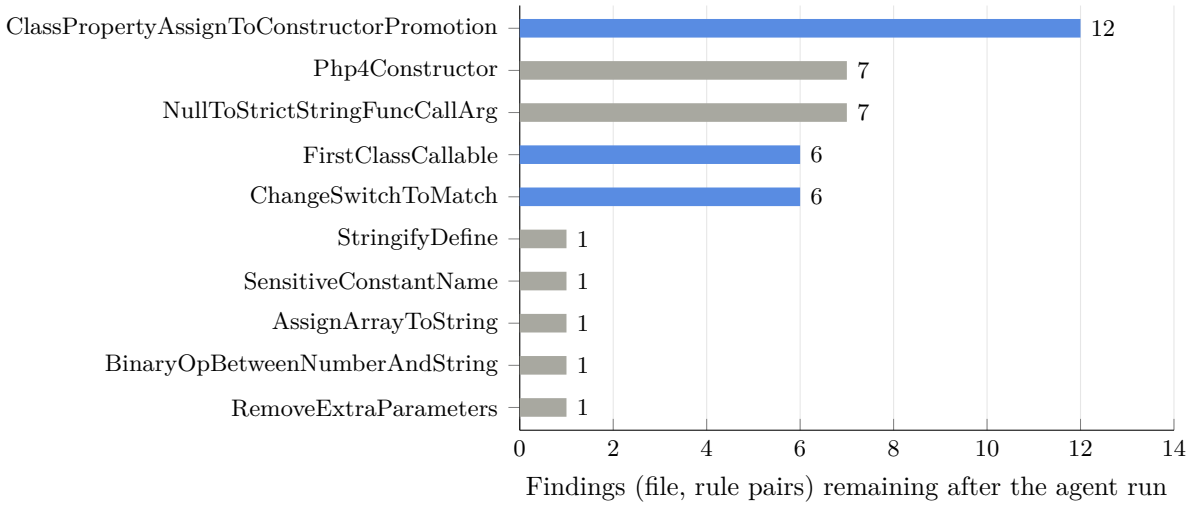


Figure 3: Rector `Php*` findings, as (file, rule) pairs, in the agent run's 100 migrated files, by rule: 43 findings across 10 rules, of which 24 are optional rewrites (blue). Grey: old-style constructors kept as compatibility wrappers (`Php4Constructor`), casts on request values (`NullToStrict`), and single cases. Source: `runs/opus55/score/per_file.csv`.

005, the agent made `$this->results[] = ...` safe where it occurs rather than changing a public default the checks read. In 066, subclasses use the argument that `RemoveExtraParametersRector` would drop. Single-shot resolves 59.5% of required findings, below the study's three strongest models, yet it fixes 10 of the 16 files with a required finding. Resolution by rule type and compatibility measure different things here, as [1] argues.

Sensitivity. Neither Opus condition has a lint failure, so the lint fallback changes nothing. Without fingerprint checks the strict single-shot result rises to 94, because its only regression (054) is a fingerprint check. The best result of any earlier model under either analysis is 69.

Significance. We apply the study's tests to 11 comparisons of our choosing and correct them jointly with Holm–Bonferroni; this family differs from the study's 21 pairwise comparisons among its own approaches. Both Opus conditions have significantly more compatible files than the unchanged code. For the strict single-shot condition the counts are 11 files gained and 1 lost, a difference of $+10$ with 95% CI $[4, 17]$ and adjusted $p = 0.025$. For the agent they are 16 and 0, a difference of $+16$ with CI $[9, 23]$ and $p = 2 \times 10^{-4}$. Both also exceed Sonnet 4 and Rector (adjusted $p \leq 0.025$). The agent's advantage over single-shot (6 files to 0 strict, 5 to 0 as delivered) has a bootstrap interval above zero ($[2, 11]$) but is not significant under the exact test after correction (adjusted $p = 0.062$; unadjusted 0.031 strict, 0.062 as delivered). On this single run, then, the agent's gain over single-shot is suggestive, not established.

5.6 File 009 is blocked by its test environment

Every approach in Table 2 scores 0 of 6 on `getid3.php` (009). Its checks construct a `getID3` object, whose constructor `includes` `getid3.lib.php` from the sandboxed WordPress tree. That library is a separate subject (001), but the harness gives each subject an unmigrated copy of the WordPress 4.0 tree, and the 4.0 library still uses curly-brace string offsets, which PHP 8 rejects at compile time. The test file defines a `getid3_lib` stub for this purpose, but it writes the stub only if the library

Table 3: Resolution by necessity, the file split by whether a file contains a required finding, and the two sensitivity analyses of [1]. Rows above the rule are computed by our script and reproduce that study's Tables III and IV; Rector's resolution is not scored. Broken counts come from the 10 compatible files with a required finding and the 73 without; fixed counts from the 16 incompatible files with one. Source: `runs/reanalysis/summary.json`.

	Resolution (%)			Files broken / fixed		Compatible files	
Approach	Req. (37)	Depr. (85)	Opt. (389)	no req. broken	req. broken / fixed	No FP	Lint fallback
Claude Sonnet 4	78.4	8.2	60.7	26	6 / 7	62	67
Gemini 2.5 Flash	78.4	22.4	57.8	33	7 / 6	53	65
Gemini 2.5 Pro	78.4	22.4	83.3	32	6 / 9	56	64
GPT-5 Codex	56.8	32.9	63.8	28	5 / 7	60	69
Llama 3.3 70B	45.9	9.4	33.7	44	7 / 1	36	45
Rector	–	–	–	6	1 / 6	82	82
Opus 5.5, single-shot (strict)	59.5	44.7	71.7	1	0 / 10	94	93
as delivered	62.2	45.9	72.8	1	0 / 11	95	94
Opus 5.5, agent	75.7	91.8	94.1	0	0 / 15	99	99

file is absent, and the sandbox always contains the real file, so the stub never takes effect. That is a defect in the harness, not in the migrations. An edit to `getid3.php` could still pass the checks by changing how it loads its own library. The 009 agent named that option, declined it as gaming the tests, and reported the stub-ordering problem instead. Excluding 009, the agent condition passes every check on all 99 remaining files.

6 Discussion

Model and harness contribute different things. Holding the protocol fixed, changing the model, together with its default thinking and sampling settings and the CLI's system prompt, moves compatible files from 59 to 93 and regressions from 126 to 1. Holding the model fixed, adding the tool loop moves compatible files from 93 to 99 and Rector resolution from 66.3% to 92.4%; on one run the file-level gain is not significant (Section 5.5). Most of the behavioral gap reported in [1] therefore closes with a newer model under the same protocol. Part of that improvement could come from exposure to later, already-modernized WordPress releases; Section 7 reports a check that finds little verbatim reuse but cannot rule it out. Prior work found the same split between generating and executing. In self-debugging [10], execution feedback added about 12 points on translation and explanation without execution about 3.5. In SWE-agent [12], the same model resolved roughly seven times more issues once it could act on a repository. The agent loop contributes the long tail of behavioral fixes and most of the gain in resolution, chiefly of deprecated findings (44.7% to 91.8%) and optional ones (71.7% to 94.1%). A single-shot model leaves about a third of the legacy patterns in place, while the agent removes most of them and explains the rest.

The agent outperforms Rector on preservation and repair. Rector is the safest single-shot option in [1]: 97.7% preservation, because its rewrites are mechanical. Its 62.6% repair shows the limit of local rules. Many legacy failures are semantic, for example `count()` on `null`, removed functions that need a behavior-equivalent replacement, and `htmlspecialchars` defaults that changed in PHP 8.1. Repairing them requires reasoning about intended behavior. The agent

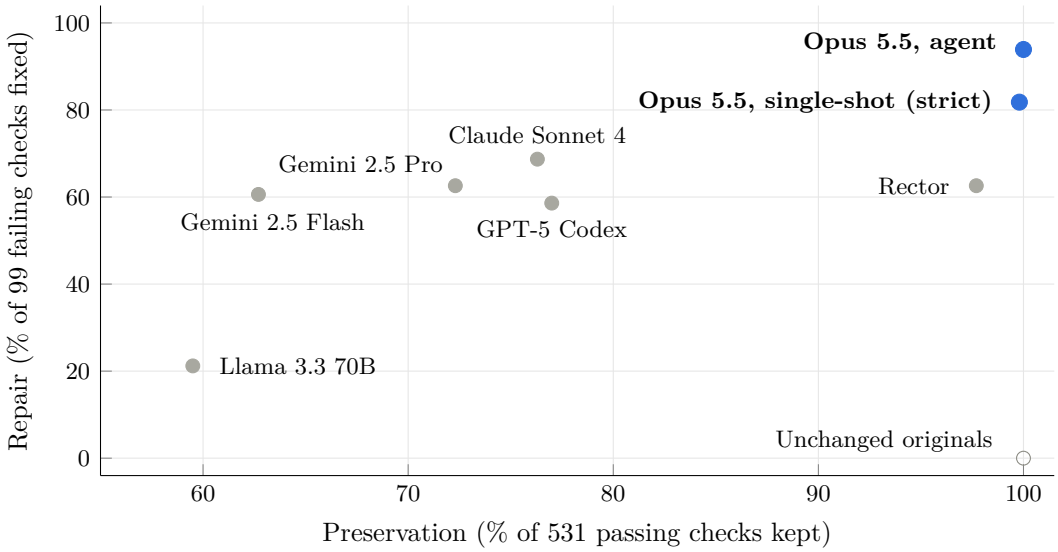


Figure 4: Preservation vs. repair per approach. The models in [1] trade one rate against the other in the lower left; Rector sits at high preservation but repairs no more than they do; both Opus conditions occupy the corner that neither rules nor older models reach. Rows from [1] are computed from the package's `summary.csv`; Opus rows from `runs/*/score/summary.json`.

reaches 100% preservation and 93.9% repair, and it uses Rector as a tool. Rector's diff shows the agent what remains, and the checks tell it which suggested rewrites are unsafe. Rule-based and LLM-based migration are complements in this workflow, not rivals. Because [1] does not score Rector's resolution against its own rules, we make no comparison with Rector on modernization.

Figure 4 places each approach by the two rates. Listing 1 shows two repairs from file 004 (`class-IXR.php`) that rules cannot make. Rector's `Php4ConstructorRector` renames `IXR_Server()` to `__construct()` and drops the old name. But WordPress's own `wp_xmlrpc_server` still calls `$this->IXR_Server($this->methods)` (`class-wp-xmlrpc-server.php`, line 159). In PHP 8 a same-name method no longer acts as a constructor. Rector's rename restores construction but makes that explicit call fail with an undefined-method error. Rector also has no rule for `count()` on `null`, which throws a `TypeError` in PHP 8. According to its final report, the agent found both by reading the callers and running the checks. That account cannot be verified, because the sessions' turn-by-turn records were not kept. The wrapper also matches, token for token, the shim WordPress itself has shipped in `class-IXR-server.php` since its PHP 4 constructor deprecations, and the single-shot run produced the same ten wrappers. This example therefore shows a repair that rules miss; it cannot distinguish reasoning from recall.

The kept wrapper is why `Php4ConstructorRector` still reports findings after the agent run (Figure 3). The agent chose a remaining Rector finding over a broken caller.

Single-shot results are a lower bound. The original condition measures how often a model can rewrite a file correctly with no ability to run it. That is a meaningful quantity: it bounds what a chat-window workflow delivers, and for the models of 2025 it showed they were not reliable when they could not observe behavior. The study's conclusion is scoped to that condition. Generalizing it to "LLMs cannot migrate code" is not supported by that study, and our results contradict it: under the same protocol for a current model, and with the file's tests for an agent.

Listing 1: File 004: original, Rector's rewrite, and the Opus 5.5 agent's output (excerpts).

```
// Original (WordPress 4.0)
function IXR_Server($callbacks = false, $data = false, $wait = false) { ... }
...
if (count($args) == 1) {          // TypeError in PHP 8 when $args is null

// Rector UP_TO_PHP_83: renames the constructor, drops IXR_Server(), leaves count() as is
function __construct($callbacks = false, $data = false, $wait = false) { ... }

// Opus 5.5 agent
public function __construct($callbacks = false, $data = false, $wait = false) { ... }

/**
 * PHP4-style constructor, kept for backward compatibility
 * (e.g. wp_xmlrpc_server calls $this->IXR_Server()).
 */
public function IXR_Server($callbacks = false, $data = false, $wait = false)
{
    self::__construct($callbacks, $data, $wait);
}
...
// (PHP < 8 semantics of count(): null counts as 0, scalars as 1)
$argCount = $args === null ? 0 : (is_countable($args) ? count($args) : 1);
if ($argCount == 1) {
```

Agreement with the original study. Our agent condition is, in effect, the staged workflow that [1] recommends in its implications: lint, load with dependencies, check behavior, and let execution feedback drive repair. Our results confirm that recommendation. They also support the study's methodological advice: measure against a no-change baseline, report preservation, regressions, and repairs separately, and read resolution by necessity rather than in aggregate. We add one suggestion: report the model generation and the evaluation condition together, since the same protocol gives opposite headlines for models a year apart.

Harness details matter at this level of performance. Once a system passes nearly every check, artifacts in the harness dominate the remaining error. File 009 accounts for every check the agent fails. Every approach fails it, because its stub never replaces the unmigrated library it is meant to stand in for. The Rector metric also depends on running each file in isolation: a directory-level run inflates the original count by 12%. Benchmarks for migration should check that each subject is passable under their own protocol, for example by including an oracle migration.

Practical implications. At a median of $0.30 and about a minute per file, an agent loop over an existing test suite is an inexpensive first pass for legacy PHP migration. Two caveats matter. The checks here are a sample of each file's contract, so passing them does not prove equivalence, and human review of the diffs remains necessary. Where no tests exist, an agent can still lint, run Rector, diff behavior against the original on PHP 7, or write its own tests. Reflexion [14] reached 91% pass@1 on HumanEval code generation with up to six unit tests that the model generated itself. That condition is weaker than ours, and we do not measure it.

7 Threats to Validity

Test visibility (construct). In the original study the checks are hidden; in our agent condition the model can read and run the same checks that score it. The agent's 99 files therefore measure what a developer with a test suite achieves, and how well a migration fits a known oracle, not how it generalizes to behavior no test covers. Its gain over single-shot cannot be attributed to execution feedback in general, as opposed to access to these particular checks. Several measures limit overfitting: instructions that forbid special-casing and environment detection, scoring against a pristine harness, and the Rector metric, which no test controls. A search of the agent outputs for test-environment sniffing (`phpunit`, the sandbox variables, `getenv`) found only an original `TMPDIR` lookup in file 015. These measures do not remove the effect. A condition with lint, Rector, and PHP but without the checks, or a held-out split of the checks, would separate the two, and we have not run either. The single-shot condition has no test exposure and still reaches 93 files.

Single run, uncontrolled sampling (conclusion). Each condition was run once per file. The CLI does not expose temperature, so we could not use the study's 0.2, as the study itself could not for GPT-5 Codex. Results may vary between runs, especially for the single-shot failures. The margins over the original models (34 or more compatible files, and over 100 fewer regressions) are significant and far larger than plausible run-to-run variation, but the six-file gap between our two conditions is not significant after correction.

Thinking and interface (internal). Opus 5.5 used adaptive extended thinking in both conditions. The client code in [13] sets no thinking or reasoning parameters, so each model in [1] ran with its provider's default. For Sonnet 4 that means no extended thinking, while both Gemini 2.5 models think by default [15] and GPT-5 Codex is a reasoning model that defaults to medium reasoning effort [16]. Our single-shot calls went through the Claude Code CLI with a one-line system prompt, not a raw API call with none. We consider both part of how the model is used today, but they make the single-shot comparison less than identical.

Environment (internal). We used PHP 8.3.36 rather than 8.3.22, and Linux on Apple's Virtualization.framework rather than Windows. The originals and the archived Sonnet 4 outputs reproduce check for check, and Rector's outputs reproduce its published totals. For three of the other earlier models the re-score shifts a few checks (at most three per model), plausibly from the PHP patch-level difference, but no compatible-file count changes. We cannot rule out similar small effects on the new outputs.

Metrics (construct). Hidden checks are a sample of each file's contract. Rector `Php*` findings are a proxy for modernization that rewards optional style changes. A low resolution rate can reflect a deliberate choice not to change behavior, as Section 5.4 shows, and aggregate resolution mixes required with optional changes (Section 5.5). The necessity mapping is our reconstruction; it reproduces every published rate, but a few rules that fire only once could arguably be grouped differently.

Training-data contamination (internal). Later WordPress releases contain PHP 7- and 8-compatible versions of many of these files, and a newer model has likely seen more of them. That could explain part of the single-shot improvement. We compared each output's added lines with the same files in WordPress 4.9.35, 6.8.12, and 7.1.4 (`harness/contamination.py`). The metric is

the share of non-trivial added lines, compared without whitespace, that appear verbatim in the later file but not in the original. Rector, which applies fixed rules and cannot be contaminated, gives the chance rate. Against 7.1.4 the agent's rate is 9.0% (Rector 7.3%), single-shot Opus 21.7%, and Sonnet 4 17.6%. Excluding lines that differ from the original only in visibility keywords or braces, the rates are 7.4%, 6.6%, 10.9%, and 9.9%. Most single-shot matches reproduce WordPress's coding-standard layout, and the excess largely disappears when only runs of whitespace are collapsed (6.9% against Rector's 5.7%). Every approach's output is less similar to the later file than the original is. The matching lines are mostly forced PHP 8 fixes and common idioms (`??`, `__DIR__`, `str_contains`, constructor shims), and no file reuses non-trivial upstream logic. Line matching cannot separate recall from convergence, and the other LLMs share some of Opus's exposure, so this bounds verbatim reuse but does not rule out contamination.

Protocol deviations (internal). Section 4.4 lists every deviation and re-launch. The single-shot response for file 014 exceeded the study's limits, and our primary result counts it as failed. In 85 agent prompts the WordPress path was blank, so the agent condition was not uniform in that respect. Agent 004 was a pilot run before the batch; it used the same prompt template and is the source of Listing 1. Re-running the agent condition in one batch with the prompt fixed, and with full session recording, would remove these caveats.

Generalization (external). The 100 files are a purposive sample from one application at one version, as [1] notes. We inherit that limit and add one model from one vendor.

Agent process not recorded (internal). The agent runs used Claude Code's headless JSON output, which records each session's final report and statistics (turns, tokens, cost, permission denials) but not the turn-by-turn conversation. Statements about how an agent reached a result therefore rest on its own final report. The results themselves do not: every outcome is verified independently by scoring the submitted file.

Evaluator independence. The agent harness, prompts, and analysis were built with the help of a Claude-based coding assistant, from the same vendor as the model evaluated. The scoring code is the original study's harness, and all outputs, final reports, and run statistics are released for inspection.

8 Conclusion

The study we revisit found that five 2025-era LLMs, asked once per file and without feedback, left legacy PHP less working than they found it, and it scoped that finding carefully. The popular reading of that study, that LLMs leave legacy code less working than they found it, does not hold for a current model under the same protocol, and does not hold at all once the model can run its output against the file's tests. Under the identical single-shot protocol and limits, Claude Opus 5.5 produces 93 compatible files against 83 for the unchanged code, and it breaks only one of the 73 compatible files that need no required change. As a coding agent with an interpreter, the file's checks, and Rector, the workflow the original study recommends, it produces 99 with no regressions. Because the agent can read the checks that score it, that figure describes migration with a test suite in hand, not blind generalization. The remaining file fails because of a defect in its test environment, not the migration. The question for LLM-based modernization is no longer whether a model can

rewrite a file blind, but how to give it the feedback a developer would have, and how to evaluate it under that condition.

Data availability. The original replication package is on Zenodo [13]. Our package contains the bash port of the runner, the workspace and tool scripts, the agent prompt, the single-shot runner, the scoring and metrics code, each agent session's final report and run statistics, all single-shot responses, the migrated files from both conditions, and per-file and summary scores. It also contains the re-analysis (`harness/reanalysis.py`: necessity mapping, sensitivity analyses, McNemar tests, and bootstrap intervals), the Rector re-score, the full-workspace tamper check, and the contamination check with its per-file results. It is public at `https://github.com/ianiv/php-modernization-agent-replication`. Local paths in the released records are replaced with `<repo>`; the records are otherwise unedited.

References

[1] Afridi Mahmud, Abid Hasan Khondaker, Sadif Ahmed, Md Nafiu Rahman, and Rifat Shahriyar. When modernization breaks working code: A single-file study of LLMs on legacy PHP. arXiv:2610.40269, 2026. URL `https://arxiv.org/abs/2610.40269`.

[2] Bill Cava. AI legacy code modernization breaks more working code than it fixes. Generative Labs, 2026. URL `https://www.generativelabs.com/insights/cisco-unlimited-ai-tokens-legacy-code-rewrite-measure-against-no-change`. October 31, 2026. Accessed 2026-11-02.

[3] Meta. Llama 3.3 70b instruct model card, 2024. URL `https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct`. Release date: December 6, 2024.

[4] OpenAI. Introducing upgrades to Codex, 2025. URL `https://openai.com/index/introducing-upgrades-to-codex/`. September 15, 2025; release of GPT-5-Codex.

[5] Anthropic. Introducing Claude 4, 2025. URL `https://www.anthropic.com/news/claude-4`. May 22, 2025.

[6] Google. We're expanding our Gemini 2.5 family of models, 2025. URL `https://blog.google/products/gemini/gemini-2-5-model-family-expands/`. June 17, 2025; 2.5 Pro and Flash generally available.

[7] Rector: Automated PHP upgrades and refactoring, v2.1.0. URL `https://github.com/rectorphp/rector`. Built on nikic/php-parser; `UP_TO_PHP_83` level set.

[8] Baptiste Rozière, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. Unsupervised translation of programming languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. URL `https://proceedings.neurips.cc/paper_files/paper/2020/hash/ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html`.

[9] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024. URL `https://arxiv.org/abs/2308.03109`.

[10] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2304.05128.

[11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.06770.

[12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

[13] Replication package for "When Modernization Breaks Working Code" (anonymized). Zenodo record 20274676, 2026. URL https://zenodo.org/records/20274676.

[14] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract.html.

[15] Google. Gemini thinking: Model defaults. URL https://aistudio.google.com/docs/thinking. Accessed 2026-11-02.

[16] OpenAI. GPT-5-Codex model documentation. URL https://developers.openai.com/api/docs/models/gpt-5-codex. Reasoning model, Responses API only; models before GPT-5.1 default to medium reasoning effort (openai-python `Reasoning.effort` docstring). Accessed 2026-11-02.